

PAGED-TIERKV: COMPACT TIERED KV CACHE MANAGEMENT FOR HIGH-CONCURRENCY LLM INFERENCE

Martin Kang, Jingwu Wang

Introduction

- Long-context, high-concurrency LLM serving is increasingly limited by **KV-cache memory**.
- As context length and active requests grow, KV storage becomes a major incremental GPU memory cost.
- This creates a systems challenge: how to reduce KV memory pressure without severely hurting inference quality.

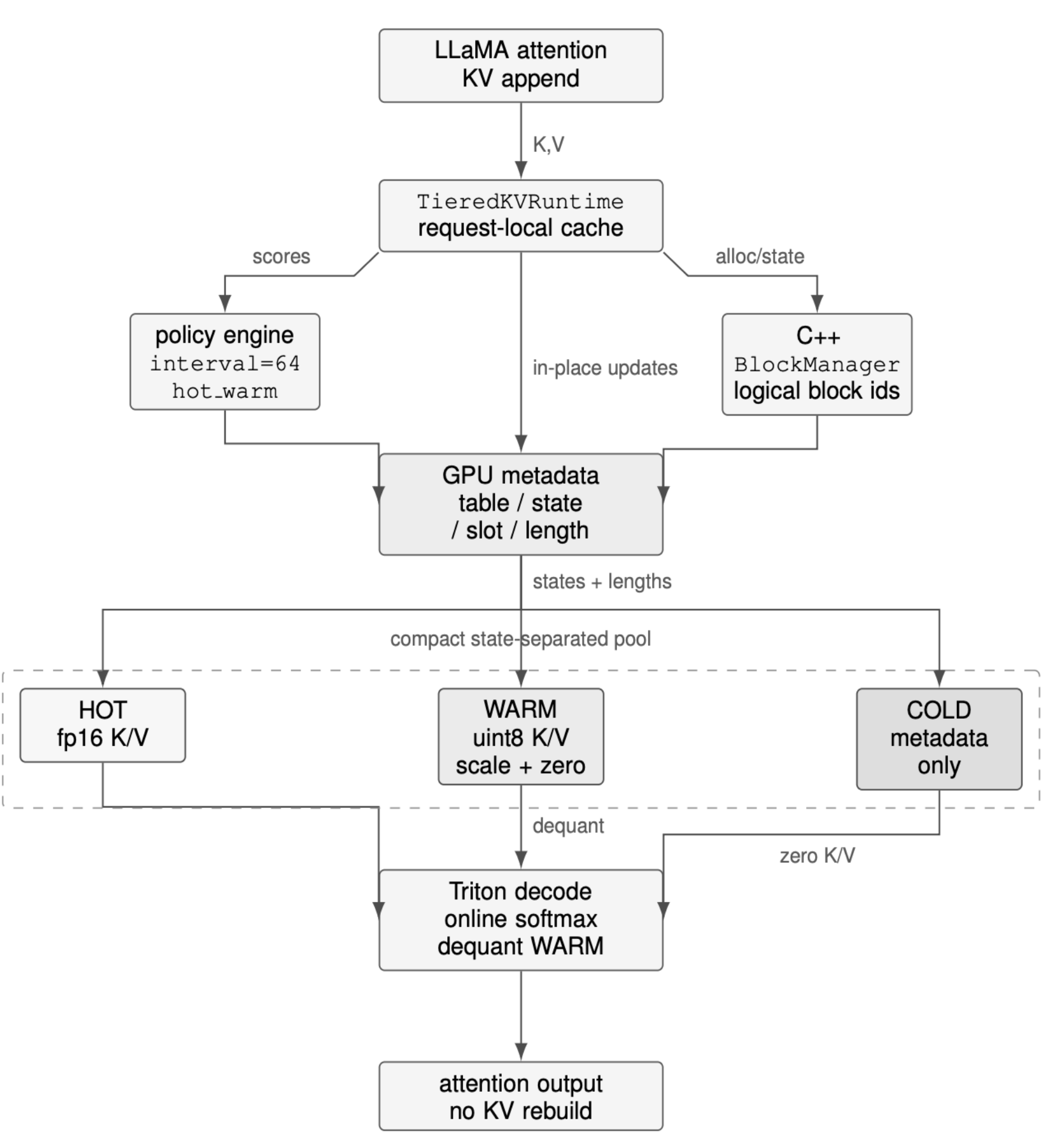
Problem Statement

- The standard baseline stores all historical KV tokens densely in fp16.
- This makes attention simple, but memory grows linearly with workload pressure and limits concurrency.
- Existing single-method solutions:
 - Quantization reduces memory, but adds overhead and may reduce fidelity.
 - Sparsification saves more memory, but may remove useful context.
- The key question is: **how to reduce physical KV memory usage while preserving enough context for accurate inference?**

Related Works

- Paged attention**: stores KV cache in fixed-size blocks and maps logical sequence positions to physical memory using a block table.
- Improves memory management flexibility, but still mainly operates on dense KV storage.
- KV quantization**: stores keys and values in lower precision, such as int8, to reduce memory footprint.
- Lowers storage cost, but introduces dequantization overhead and possible approximation error.
- KV sparsification**: drops less important tokens or cache blocks entirely to save memory.
- Achieves high compression, but may lose important information needed for retrieval

Method



- Paged-TierKV**, a tiered KV-cache system that combines paging, selective quantization, and selective sparsification.
- KV history is divided into fixed-size blocks and assigned to three tiers:
 - HOT**: fp16, high fidelity
 - WARM**: int8 + metadata, lower memory
 - COLD**: metadata only, no dense KV storage
- The design aims to preserve important context while reducing physical KV memory usage under high concurrency.

Evaluation

- Evaluated on TinyLlama-1.1B-Chat with 1900-token prompts on an NVIDIA A10G.
- Quality measured on 50 Qasper examples from LongBench.
- Aggressive COLD eviction causes major quality loss.
- HOT+WARM gives the best quality-memory tradeoff:
 - 10.67 F1 vs. 11.17 dense baseline**
 - 1.68× physical KV compression**

Table 1. Qasper-50 quality and memory summary at 1900 input tokens.

Configuration	Peak MB	F1	tok/s	Phys. comp.
Baseline	2238.56	11.17	53.29	—
Pure Quantization	2220.81	10.75	16.27	1.75×
Pure Sparsification	2198.53	0.93	32.94	29.99×
Paged-TierKV, budget 4	2220.46	10.55	16.44	1.77×
Paged-TierKV, budget 8	2222.87	10.67	16.75	1.68×

Table 2. Tier-mode ablation. HOT+WARM is the final default because it preserves quality while compressing physical KV.

Mode	F1	tok/s	HOT/WARM/COLD	Phys. comp.
HOT-only	11.16	23.53	2662/0/0	0.99×
HOT+WARM	10.67	12.77	220/2442/0	1.68×
HOT+WARM+COLD	1.47	19.80	220/704/1738	4.40×

Evaluation (Scaling)

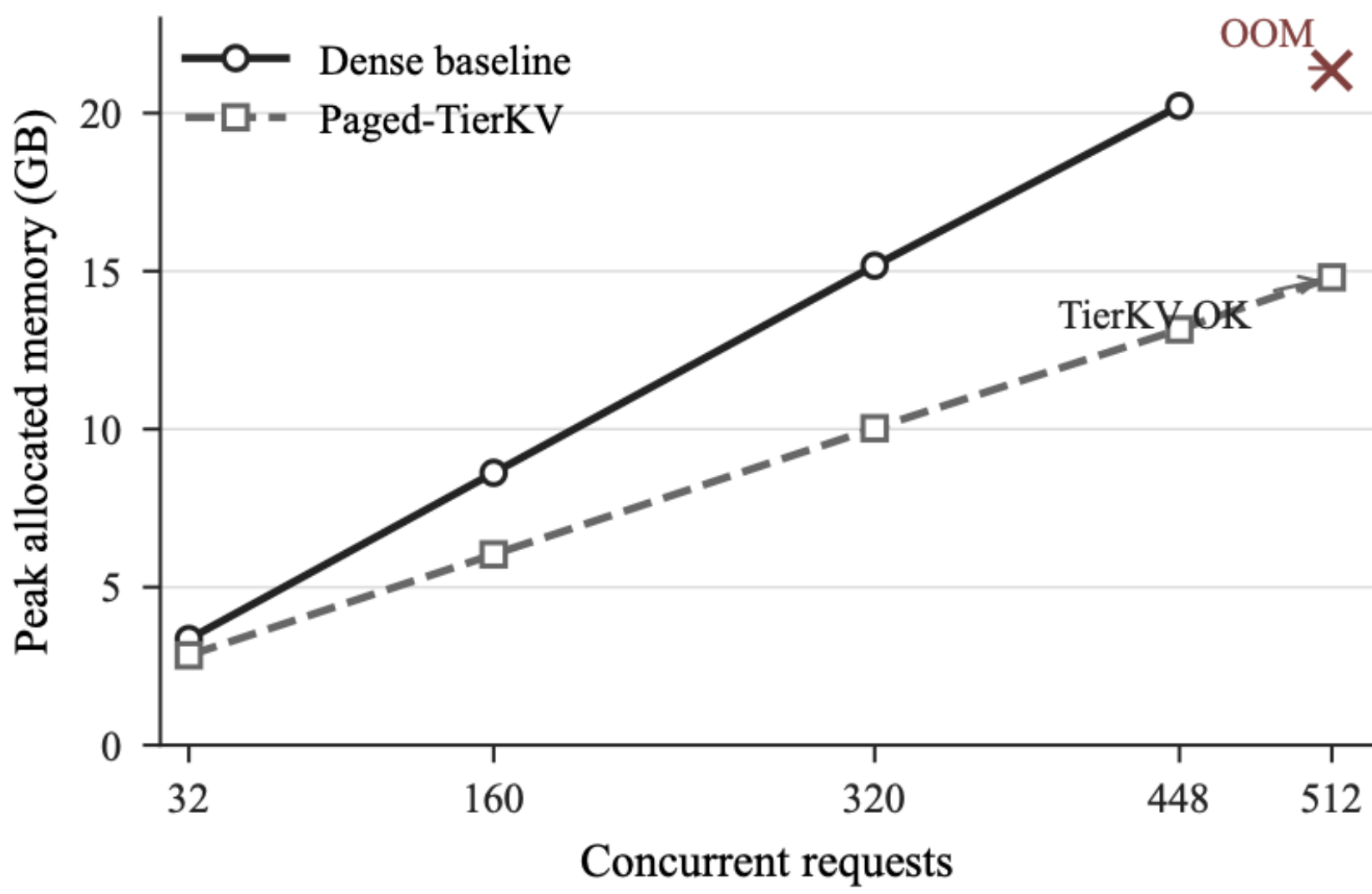


Figure 2. Peak memory scaling for the TinyLlama high-concurrency benchmark.

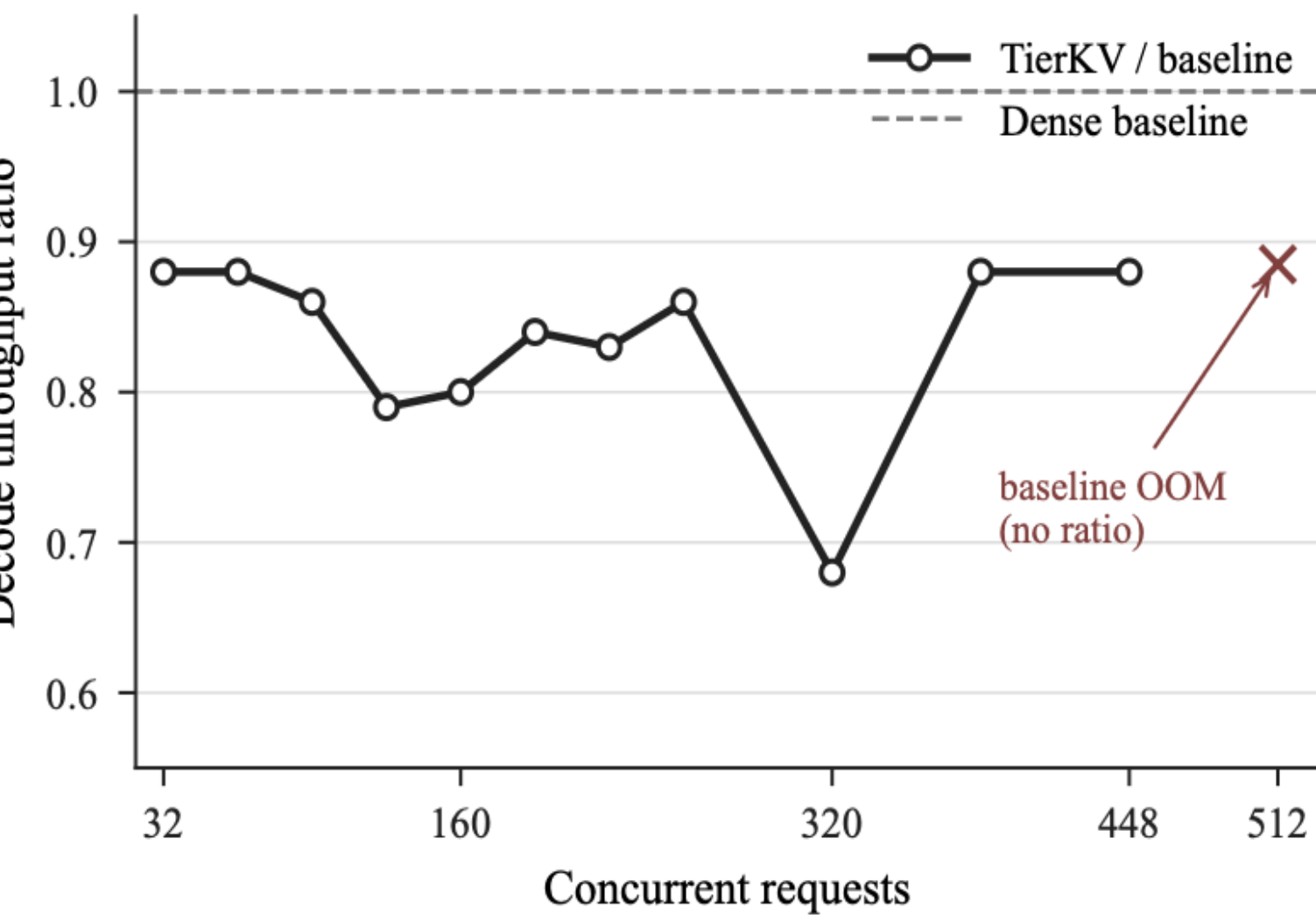


Figure 3. Throughput ratio for the TinyLlama high-concurrency benchmark.

Table 4. Peak-memory slope summary. Lower MB/request means better memory scaling as concurrency grows.

Range	Baseline	TierKV
TinyLlama 32–160	42.06	25.55
TinyLlama 192–320	43.01	26.08
TinyLlama 384–448	43.26	26.90
Vicuna 1024, 1–12	536.00	339.63
Vicuna 1536, 1–8	791.65	479.32

512 concurrent requests:
Dense baseline OOM;
Paged-TierKV succeeded

448 concurrent requests:
Peak allocated memory
drops from 21 GB to 13
GB

Throughput is 0.7 – 0.9X
baseline throughput

TinyLlama: Peak
memory slope drops to
25 MB/request

Vicuna 7B extension:
Peak memory drops to
300 MB/request

Conclusion

- Paged-TierKV converts logical KV compression into real physical memory savings through **state-specific storage**.
- Main Strength: memory scaling
- Main limitation: throughput remains below dense baseline because of dequantization and metadata overhead.